ABSTRACTOR

A Dissertation submitted in partial fulfillment of the requirement for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Of

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

By

Mr. SYED MAFAAZ

4SN24MC037

Under the Guidance of

Dr. DHEERAJ.H

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

SRINIVAS INSTITUTE OF TECHNOLOGY

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi,

Recognized by Govt. of Karnataka)

Mangalore -574143, Karnataka

DECEMBER 2025

ABSTRACTOR

A Dissertation submitted in partial fulfillment of the requirement for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

Of

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

By

Mr. SYED MAFAAZ

4SN24MC037

Under the Guidance of

Dr. DHEERAJ.H
Associate Professor
Department of MCA
SIT, Mangalore

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

SRINIVAS INSTITUTE OF TECHNOLOGY

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi,

Recognized by Govt. of Karnataka)

Mangalore -574143, Karnataka

SRINIVAS INSTITUTE OF TECHNOLOGY

(NAAC Accredited)

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

Valachil, Mangalore-574143

(Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi, Recognized by Govt.of Karnataka)

CERTIFICATE

This is to certify that, ***Mr. SYED MAFAAZ*** *bearing* ***USN 4SN24MC037*** *has successfully completed his third semester project work entitled "****ABSTRACTOR "*** *as a partial fulfillment for the award for Master of Computer Applications degree, during the academic year 2025-2026.*

Dr. Dheeraj Hebri
Associate Professor
Department of MCA
SIT, Mangalore

Dr. Shashidhar Kini K
Head of Department
Department of MCA
SIT, Mangalore

Dr. Shrinivasa Mayya D
Principal
SIT, Mangalore

Date:

Examiners **Signature**

1.

2.

DECLARATION

I, **Mr. SYED MAFAAZ**, student of 3rd Semester MCA, **Srinivas Institute of Technology, Valachil**, bearing USN: **4SN24MC037**, hereby declare that I have carried out the project entitled "**ABSTRACTOR**" under the supervision of **Dr. Dheeraj H**, Associate Professor, Department of Master of Computer Applications, and submitted the same in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications during the academic year 2025–2026, from Visvesvaraya Technological University, Jnana Sangama, Belagavi.

This work which is being presented in the project report submitted to the Department of MCA from Srinivas Institute of Technology, Valachil, Mangalore, is an authentic record of project work.

Place: Mangalore
Date:

Mr. Syed Mafaaz
Signature:

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the guidance, support, and encouragement of several individuals to whom I express my sincere gratitude. First and foremost, I would like to express my heartfelt thanks to my project guide, **Dr. Dheeraj H**, Associate Professor, Department of Master of Computer Applications, for his constant guidance, valuable suggestions, and continuous encouragement throughout the course of this project. His academic expertise and insightful feedback played a significant role in shaping the project effectively.

I express my sincere gratitude to the Head of the Department**, Dr. Shashidhar Kini K**, Department of Master of Computer Applications, Srinivas Institute of Technology, for providing a supportive academic environment and for his encouragement throughout the project period.

I am also grateful to the Principal, **Dr. Shrinivasa Mayya D**, Srinivas Institute of Technology, for his encouragement and support, which greatly contributed to the successful completion of this project.

I extend my appreciation to all the faculty members and non-teaching staff of the Department of Master of Computer Applications for their cooperation, guidance, and support throughout the course of the project. I would also like to thank the management of Srinivas Institute of Technology for providing the necessary infrastructure and facilities to carry out this work.

Finally, I express my heartfelt gratitude to my parents for their constant encouragement and moral support. I also thank my friends and classmates who directly or indirectly contributed to the successful completion of this project.

INDEX

CHAPTER NUMBER	TITLE	PAGE NUMBER
1	**INTRODUCTION**	**1-4**
	1.1 PROJECT DESCRIPTION	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVE OF THE STUDY	3
2	**LITERATURE REVIEW**	**5-10**
	2.1 EXISTING SYSTEM AND PROPOSED SYESTEM	5
	2.1.1 EXISTING SYSTEM	7
	2.1.2 PROPOSED SYSTEM	8
	2.2 FEASIBILITY STUDY	10
	2.2.1 TECHNICAL FEASIBILTY	9
	2.2.2 OPERATIONAL FEASIBILITY	10
	2.2.3 ECONOMIC FEASIBILTY	10
3	**SOFTWARE REQUIREMENT SPECIFICATION**	**11-18**
	3.1 PURPOSE	11
	3.2 SCOPE	13
	3.3 FUNCTIONAL REQUIREMENTS	15
	3.4 NON-FUNCTIONAL REQUIREMENTS	17
	3.5 HARDWARE AND SOFTWARE REQUIREMENTS	18
	3.5.1 HARDWARE REQUIREMENTS	18
	3.5.2 SOFTWARE REQUIREMENTS	18
4	**METHODOLOGY**	**19-26**
	4.1 INTRODUCTION	19
	4.2 DATA SOURCE	20
	4.3 PREPROCESSING	22
	4.4 EXTRACTION AND PREPARATION	23

	4.5 DATA EXPLORATION AND ANALYSIS	23
	4.6 WORKFLOW DIAGRAM	26
5	**MODEL DETAIL AND IMPLEMENTATION**	**27-35**
	5.1 ALGORITHMS	27
	5.1.1 BART TRANSFORMER MODEL	29
	5.1.2 TF-IDF EXTRACTIVE ALGORITHM	30
	5.2 MODEL BUILDING	31
	5.3 DATA INPUT	32
	5.4 USER INTERFACE	35
6	**TESTING**	**36-38**
	6.1 INTRODUCTION	36
	6.2 TEST CASES	37
	6.3 TEST RESULTS	38
7	**CONCLUSION**	**39-40**
8	**FUTURE ENHANCEMENTS**	**41-42**
	8.1 MULTI-STYLE SUMMARIES	41
	8.2 MULTI-LANGUAGE SUPPORT	41
	8.3 SUMMARIZATION EXPLAINABILTY	42
	8.4 AUTHENTICATION AND USER ACCOUNTS	42
	8.5 IMPROVED LAYOUT AND NAVIGATION	42
APPENDIX A	**BIBLIOGRAPHY**	**43**
APPENDIX B	**USER MANUAL**	**46**

LIST OF FIGURES

FIGURE NUMBER	FIGURES	PAGE NUMBER
4.1	System Architecture Diagram	24
4.2	Workflow Diagram	24
5.1	BART Transformer Architecture	28
5.2	Summarization Flowchart	30
5.3	Text Chunking Algorithm	30
5.4	Home Page Interface	32
5.5	File Upload Interface	33
5.6	Summary Result Page	33
5.7	History Page	34
5.8	Dataflow Diagram	34

ABSTRACT

In the present digital era, the exponential growth of textual data across articles, reports, research papers, and online content has created a critical need for efficient information condensation techniques. Manual summarization is time-consuming, error-prone, and impractical for large volumes of text. To address this challenge, this project presents ABSTRACTOR – An AI-Powered Text Summarization Tool, a web-based system designed to automatically generate concise, meaningful summaries from large textual inputs using modern Natural Language Processing (NLP) techniques.

The proposed system supports both extractive and abstractive summarization approaches, with a primary focus on deep learning–based abstractive summarization using the BART (Bidirectional and Auto-Regressive Transformer) model. The system is capable of processing plain text as well as document files such as PDF, DOCX, and TXT, making it suitable for real-world academic and professional use cases. A robust text chunking algorithm is implemented to handle large documents by splitting them into manageable segments while preserving contextual continuity.

The architecture follows a three-tier design, consisting of a React-based frontend for user interaction, a Flask backend for request handling and processing, and a service layer responsible for text extraction, preprocessing, summarization, and fallback mechanisms. When external AI services are unavailable or fail to respond, the system seamlessly switches to a TF-IDF-based extractive summarization method, ensuring reliability and uninterrupted service. Additionally, the application maintains summary history using browser storage, enhancing usability and user experience.

Overall, ABSTRACTOR demonstrates how transformer-based language models can be effectively integrated into a scalable web application to deliver fast, accurate, and user-friendly text summarization. The system significantly reduces reading time while preserving essential information, making it a valuable tool for students, researchers, and professionals dealing with large textual datase

CHAPTER 1

INTRODUCTION

1.1 PROJECT DESCRIPTION

Abstractor AI is an advanced web-based application that leverages cutting-edge artificial intelligence and natural language processing technologies to provide intelligent text summarization services. The system is designed to address the growing challenge of information overload in the digital age by automatically condensing lengthy documents into concise, meaningful summaries while preserving the essential information and context.

The application supports multiple input methods, including direct text entry and file uploads in various formats such as PDF (Portable Document Format), TXT (Plain Text), and DOCX (Microsoft Word Document). This flexibility ensures users can process documents from diverse sources without requiring manual format conversion or specialized software.

At its core, Abstractor AI employs a sophisticated dual-algorithm approach. The primary method utilizes Facebook BART (Bidirectional and Auto-Regressive Transformers) model for abstractive summarization, which generates human-like summaries by understanding context and paraphrasing content. When AI services are unavailable, the system intelligently falls back to TF-IDF (Term Frequency-Inverse Document Frequency) based extractive summarization, ensuring uninterrupted service delivery.

The system architecture follows a modern client-server model with clear separation of concerns. The frontend is built using React 19.2.1, providing an intuitive and responsive user interface with features such as drag-and-drop file upload, real-time character counting, and visual progress indicators. The backend utilizes Flask 3.0.0, a lightweight Python web framework, to handle API requests, text processing, and external AI service integration.

Key features include adaptive length control through three configurable presets (short, medium, and long summaries), comprehensive history management with persistent local storage for up to 20 entries, robust error handling with user-friendly feedback messages, and optimized performance targeting processing times under 30 seconds for typical documents. The application is designed to serve diverse user groups including researchers conducting literature reviews, business professionals analyzing reports, students condensing study materials, and content creators extracting key points from articles.

1.2 PROBLEM STATEMENT

In the contemporary digital landscape, individuals and organizations encounter an unprecedented volume of textual information daily. Research papers, technical reports, business documents, news articles, and academic literature are produced at an exponential rate, creating significant challenges for professionals who need to stay informed and make evidence-based decisions.

The primary problems identified include:

1. Time Constraint and Information Overload: Professionals across various domains spend excessive time reading lengthy documents to extract relevant information. A typical research paper may contain 8,000-12,000 words, requiring 30-45 minutes to read thoroughly. Business reports often exceed 50 pages, demanding hours of focused attention. This time investment significantly reduces productivity and limits the volume of information that can be processed effectively.

2. Quality Inconsistency in Manual Summarization: When individuals attempt to summarize documents manually, results vary based on personal skills, domain knowledge, and time availability. Important details may be overlooked, biases can influence content selection, and the summarization quality lacks standardization. This inconsistency makes it difficult to compare information across multiple sources or maintain organizational knowledge bases.

3. Format Diversity and Processing Complexity: Documents exist in numerous formats including PDF, Microsoft Word, plain text, HTML, and proprietary formats. Processing these diverse formats typically requires multiple tools and manual conversion steps. PDF files may contain scanned images requiring OCR (Optical Character Recognition), Word documents include complex formatting that complicates text extraction, and different tools may produce inconsistent results from the same source.

4. Accessibility and Cost Barriers: Existing commercial summarization services often impose significant barriers. Subscription-based models charge monthly fees ranging from $10 to $100+ depending on usage limits. Per-document pricing can accumulate quickly for regular users. Many services restrict features in free tiers, limiting summary length, processing speed, or file size. These financial barriers make professional-grade summarization tools inaccessible to individual users, students, and small organizations.

5. Privacy and Security Concerns: Cloud-based summarization services require uploading documents to external servers, raising concerns about data privacy and confidentiality. Sensitive business documents, proprietary research, and personal information face exposure risks during transmission and processing. Organizations with strict data

governance policies may be prohibited from using external summarization services, limiting their options to inadequate in-house solutions.

6. Lack of Customization and Control: Most existing tools provide limited options for controlling summary characteristics. Users cannot specify desired summary length, adjust detail levels for different purposes, or configure processing parameters to match specific requirements. This one-size-fits-all approach produces suboptimal results for diverse use cases ranging from quick overviews to detailed executive summaries.

These interconnected challenges create a clear need for an intelligent, accessible, and reliable document summarization solution that can process multiple formats, generate high-quality summaries quickly, provide customization options, ensure data privacy, and remain financially accessible to all user segments.

1.3 OBJECTIVES

The primary objectives of the Abstractor AI project are:

1. Develop an AI-powered text summarization system that accurately condenses lengthy documents while preserving essential information, context, and meaning through advanced natural language processing techniques.

2. Implement a robust dual-algorithm approach combining state-of-the-art abstractive summarization using Facebook BART transformer model with reliable extractive summarization using TF-IDF statistical methods to ensure consistent service availability regardless of external API status.

3. Support comprehensive document format compatibility by implementing native support for PDF, TXT, and DOCX formats with robust text extraction capabilities that handle various document structures, encodings, and complexities without requiring manual conversion.

4. Provide flexible summarization customization through three configurable length presets (short: 80 characters maximum, medium: 150 characters maximum, long: 250 characters maximum) that cater to different use cases including quick overviews, balanced summaries, and detailed abstracts.

5. Design an intuitive, user-friendly web interface using modern React framework that requires no technical expertise, provides clear visual feedback during all operations, implements drag-and-drop file upload functionality, and maintains responsive design across desktop, tablet, and mobile devices.

6. Implement comprehensive history management with persistent browser-based local storage that tracks up to 20 recent summarizations with complete metadata including

timestamps, source types, original text previews, generated summaries, and processing parameters for future reference and analysis.

7. Ensure optimal performance targeting processing times under 30 seconds for typical documents (1,000-5,000 characters) through efficient text chunking algorithms, optimized API integration, and intelligent resource management.

8. Build a scalable system architecture based on stateless design principles that support horizontal scaling, enable independent deployment of frontend and backend components, facilitate load balancing through standard reverse proxy configurations, and accommodate future feature additions without significant restructuring.

9. Implement comprehensive error handling and validation mechanisms that provide helpful user feedback for validation failures, network errors, processing timeouts, and file extraction issues while preventing system failures and maintaining operational continuity.

10. Create extensive documentation covering system architecture, API specifications, deployment procedures, troubleshooting guides, and user manuals to facilitate deployment, maintenance, future development, and user adoption across diverse technical skill levels.

CHAPTER 2

LITERATURE SURVEY

2.1 EXISTING SYSTEM AND PROPOSED SYSTEM

2.1.1 EXISTING SYSTEM

Current text summarization solutions in the market can be categorized into several distinct types, each with specific characteristics, advantages, and limitations that impact their suitability for different use cases and user requirements.

Commercial Cloud-Based Services:

Services such as QuillBot, TLDR This, SummarizeBot, and Resoomer offer summarization through web interfaces or RESTful APIs. These platforms typically employ advanced AI models but impose subscription fees ranging from $10 to $100 per month depending on usage limits and feature access. The pricing models include per-document charges, monthly character limits, and tiered subscription plans with feature restrictions in free tiers. While these services often provide good quality summaries through proprietary algorithms, several significant concerns limit their applicability. Data privacy issues arise as documents must be uploaded to external servers for processing, raising confidentiality concerns for sensitive materials. Usage restrictions include document size limits, processing quotas, and API rate limiting. Cost barriers make regular use expensive for individuals, students, and small organizations. Service dependency means functionality is unavailable during outages or network disruptions.

Rule-Based and Statistical Systems:

Traditional summarization tools rely on rule-based approaches and simple statistical methods such as word frequency analysis, sentence position scoring, and keyword extraction. These systems implement algorithms like basic TF-IDF (Term Frequency-Inverse Document Frequency) without advanced natural language understanding, simple sentence ranking based on word occurrence statistics, and position-based importance scoring favoring introduction and conclusion sentences. Advantages include fast processing speed, deterministic and reproducible results, and minimal computational requirements enabling local execution. However, significant limitations affect output quality: summaries often lack coherence and readability, contextual nuances and semantic relationships are missed, extracted sentences may be disjointed and awkward, and the approach struggles with complex documents requiring synthesis of information across multiple sections.

Desktop Applications and Office Suite Features:

Standalone applications and built-in features like Microsoft Word AutoSummarize provide offline summarization capabilities. These tools offer the advantage of local processing without internet connectivity requirements and integration with existing document workflows. However, they suffer from limited format support restricted to native document types, outdated algorithms that predate modern NLP advances, lack of customization preventing users from adjusting summary characteristics, and absence of updates or improvements as development has ceased for many legacy features.

Browser Extensions and Add-ons:

Extensions such as SMMRY, Resoomer, and various Chrome/Firefox add-ons offer convenient summarization of web content directly from the browser. While convenient for online reading, these tools have restricted functionality limited to visible web content, cannot process uploaded documents or local files, typically use basic algorithms rather than advanced AI, and lack history management or advanced features found in dedicated applications.

Open-Source Solutions:

Open-source libraries and tools like Sumy, Gensim, and various Python NLP libraries provide programmatic access to summarization algorithms. While these offer flexibility and customization potential, they require significant technical expertise for setup and use, lack user-friendly interfaces for non-programmers, demand manual integration and maintenance effort, and provide limited support or documentation compared to commercial products.

Limitations Summary:

High cost barriers through subscription models or pay-per-use pricing structures that accumulate quickly

Privacy concerns with cloud-based processing of potentially sensitive or confidential documents

Limited format support requiring manual conversion between document types

Lack of customization in summary length, detail level, and stylistic characteristics

Inconsistent quality across different document types, subject areas, and content structures

No offline capabilities requiring constant internet connectivity for operation

Poor user experience with complex interfaces requiring technical knowledge

Absence of history management and tracking features for reviewing past summarizations

Limited or no API access for programmatic integration into workflows

Inadequate error handling and fallback mechanisms when processing fails

2.1.2 PROPOSED SYSTEM

Abstractor AI addresses the comprehensive range of limitations identified in existing systems through an innovative, integrated approach that combines cutting-edge AI technology with practical usability considerations.

Hybrid AI Architecture:

The system implements a sophisticated dual-algorithm approach that combines two complementary summarization methods. The primary method employs Facebook BART (Bidirectional and Auto-Regressive Transformers), a state-of-the-art transformer model specifically trained for abstractive summarization on the CNN/DailyMail dataset. This model generates human-like summaries through natural language generation, capable of paraphrasing and synthesizing information rather than merely extracting sentences. When external AI services are unavailable due to network issues, API rate limits, or service outages, the system automatically switches to an intelligent TF-IDF based extractive summarization algorithm that analyzes word frequencies and importance scores to select the most representative sentences. This fallback ensures users always receive results while maintaining reasonable quality standards.

Comprehensive Format Support:

Native processing capabilities for PDF (Portable Document Format), DOCX (Microsoft Word Document), and TXT (Plain Text) formats eliminate the need for manual conversion workflows. Specialized extraction libraries including PyPDF2 for PDF processing and python-docx for Word documents ensure accurate text extraction while preserving document structure and readability. The system handles various PDF versions from 1.3 onwards, processes Office 2007+ DOCX files, supports multiple text encodings with UTF-8 as primary, and gracefully handles edge cases like password-protected files and corrupted documents with informative error messages.

Flexible Customization Options:

Three carefully designed length presets provide users with control over summary detail levels appropriate for different contexts. The short preset (maximum 80 characters, minimum 25 characters) generates concise overviews ideal for quick scanning, processes 3 text chunks, and targets 2 sentences in the output. The medium preset (maximum 150 characters, minimum 50 characters) delivers balanced summaries suitable for general use,

processes 5 text chunks, and targets 4 sentences. The long preset (maximum 250 characters, minimum 80 characters) produces detailed abstracts for comprehensive review, processes 7 text chunks, and targets 6 sentences. Each preset is optimized with specific parameters for token counts, sentence numbers, and chunk processing to deliver appropriate results efficiently.

Modern User Interface Design:

The React-based frontend provides an intuitive, responsive interface that adapts seamlessly across desktop, tablet, and mobile devices. Key interface features include drag-and-drop file upload with visual feedback for drag-over states, real-time character counting with color-coded status indicators showing valid (green), warning (orange), and empty (gray) states, clear visual progress indicators during processing with loading animations, and comprehensive error messages that explain issues and suggest solutions. The design emphasizes simplicity and accessibility, requiring no technical knowledge from users while providing power users with keyboard shortcuts and efficient workflows.

Advantages of the Proposed System:

Free and open-source with no subscription fees, usage limits, or hidden costs

Privacy-focused design with client-side storage and optional local deployment

High-quality AI-generated summaries using transformer models when available

Reliable fallback mechanism ensures continuous operation without service interruptions

Multi-format support without requiring external conversion tools

Customizable output through well-designed length presets

Fast processing with optimized chunking algorithms targeting under 30 seconds

Scalable architecture supporting concurrent users and horizontal scaling

Comprehensive error handling providing clear feedback and recovery options

Extensible design facilitating future enhancements without major restructuring

2.2 FEASIBILITY STUDY

2.2.1 TECHNICAL FEASIBILITY

Technical feasibility analysis examines whether the project can be successfully implemented using available technologies, development resources, and technical expertise within reasonable time and budget constraints.

Technology Maturity and Availability:

All required technologies are mature, well-documented, and freely available for use. React 19.x represents the latest stable version of the most popular frontend framework, backed by Facebook/Meta with continuous development and extensive community support. Flask 3.x provides a production-ready Python web framework with over a decade of real-world usage across millions of applications. The Hugging Face Transformers library offers reliable access to state-of-the-art NLP models with comprehensive documentation, active maintenance, and proven scalability. Supporting libraries including PyPDF2, python-docx, and Requests have established track records in production environments.

Development Resource Accessibility:

The project utilizes exclusively open-source tools and libraries, eliminating licensing costs and legal constraints. Development environments such as Visual Studio Code, PyCharm Community Edition, and Sublime Text are freely available with extensive plugin ecosystems. Version control through Git and hosting on platforms like GitHub, GitLab, or Bitbucket provide robust collaboration capabilities without cost. Cloud hosting options including AWS Free Tier, Google Cloud Free Tier, Heroku free dynos, and DigitalOcean provide adequate resources for development and initial deployment without upfront investment.

Conclusion - Technical Feasibility:

The project demonstrates strong technical feasibility with all components proven in production environments, architecture patterns well-established in industry practice, and integration points clearly defined and tested by the community.

2.2.2 OPERATIONAL FEASIBILITY

Operational feasibility assesses whether the system will be effectively utilized, accepted by target users, and maintainable over its intended lifecycle.

User Acceptance and Usability:

The system addresses a clear, widespread need for efficient document summarization across multiple user segments. The user interface follows familiar web application patterns including standard form elements, recognizable icons and buttons, conventional drag-and-drop interactions, and expected keyboard navigation, minimizing the learning curve for new users. Immediate feedback through visual indicators, progress animations, and status messages helps users understand system state and available actions.

Conclusion - Operational Feasibility:

The system demonstrates high operational feasibility with strong user acceptance potential, realistic performance expectations, adequate reliability mechanisms, and maintainable architecture supporting sustained operation.

2.2.3 ECONOMIC FEASIBILITY

Economic feasibility evaluates the cost-benefit relationship, financial viability, and return on investment potential for the project from both development and deployment perspectives.

Development Cost Analysis:

Direct costs are minimal as all core technologies (React, Flask, Python libraries) are open-source and free. Development tools including code editors, version control, and local testing environments are available at no cost. The primary investment is developer time, which varies based on expertise level and project timeline but is significantly lower than commercial alternatives due to reusable components and extensive library support.

Return on Investment:

For individual users, time saved on document review quickly justifies any minimal operational costs. A professional reading 10 documents per week might save 5-10 hours monthly, translating to significant productivity gains. For organizations, benefits include faster decision-making through quicker information access, improved research efficiency across teams, and enhanced competitive intelligence gathering.

Conclusion - Economic Feasibility:

The project demonstrates strong economic feasibility with minimal development and operational costs, clear benefits providing positive ROI for users, and cost-effectiveness maintained through open-source technologies and efficient cloud platforms.

CHAPTER 3
SOFTWARE REQUIREMENT SPECIFICATION

3.1 PURPOSE

The purpose of this Software Requirement Specification (SRS) document is to provide a comprehensive and detailed description of the Abstractor AI system, encompassing all aspects of functionality, performance, design constraints, and external interfaces. This document serves as the authoritative reference for all stakeholders including developers, testers, project managers, quality assurance teams, and end-users throughout the software development lifecycle.

The SRS establishes a clear, unambiguous understanding of system capabilities and requirements, facilitating effective communication among team members and stakeholders. It provides the foundation for project planning activities including effort estimation, resource allocation, timeline development, and milestone definition. The document serves as the basis for validation and verification activities, ensuring the final product meets specified requirements and user expectations.

Additionally, this SRS supports change management by providing a baseline against which proposed modifications can be evaluated. It facilitates maintenance activities by documenting system behavior and constraints. The specification enables future enhancement planning by clearly defining current capabilities and identifying extension points. For procurement and contracting scenarios, it provides a definitive statement of deliverables and acceptance criteria.

3.2 SCOPE

Abstractor AI is a comprehensive web-based document summarization system that provides intelligent text condensation services through the application of artificial intelligence and natural language processing technologies. The system scope encompasses the following capabilities and components:

In-Scope Functionality:

Text Input Processing: Accept direct text input through textarea interface with minimum 100 character validation, real-time character counting with color-coded status indicators (green: valid, orange: warning, gray: empty), maximum practical length support up to 50,000 characters, and clear input functionality for quick reset.

File Upload Support: Accept file uploads via drag-and-drop or file browser selection for PDF (Portable Document Format), TXT (Plain Text), and DOCX (Microsoft Word Document) formats up to 16MB in size. Validate file types through extension and MIME

type checking, display uploaded filename with option to remove and select different file, and provide visual feedback during drag-over states.

AI-Powered Summarization: Utilize Facebook BART (Bidirectional and Auto-Regressive Transformers) transformer model via Hugging Face Inference API for abstractive summarization. Generate human-like summaries that paraphrase and synthesize information, process text in chunks to handle long documents exceeding model token limits, and configure parameters based on selected length preset.

Intelligent Fallback Mechanism: Automatically switch to extractive TF-IDF (Term Frequency-Inverse Document Frequency) based summarization when AI services are unavailable due to network errors, API rate limits, service outages, or timeout conditions. Ensure users always receive summarization results regardless of external service status.

Summary Length Customization: Provide three carefully designed preset options - Short (maximum 80 characters, minimum 25, targeting 2 sentences), Medium (maximum 150 characters, minimum 50, targeting 4 sentences), and Long (maximum 250 characters, minimum 80, targeting 6 sentences). Adjust processing parameters including chunk count processed based on preset selection.

Real-Time Processing Feedback: Display loading indicators with animated spinners during summarization processing, show progress messages indicating current operation stage, disable input controls during processing to prevent conflicting requests, and provide estimated processing time when available.

History Management System: Automatically save successful summarizations to browser localStorage with metadata including timestamp, source type (text or file), original text preview (first 500 characters), complete summary text, and length preset used. Store last 20 entries with chronological ordering, provide dedicated history view with list display, enable detailed view modal for any history entry, and offer bulk clear functionality to delete all history.

Responsive Web Interface: Deliver user interface accessible from desktop computers, tablets, and mobile devices with automatic layout adaptation. Implement touch-friendly controls for mobile use, maintain readable text sizing across devices, and ensure all functionality available regardless of screen size.

RESTful API Communication: Establish JSON-based communication between frontend and backend components using HTTP POST requests. Implement /api/summarize endpoint for text input, /api/summarize/file endpoint for file uploads, and /api/health endpoint for service status checks.

Comprehensive Error Handling: Validate all user inputs before processing with clear error messages for validation failures. Handle network errors and timeouts gracefully with retry suggestions. Provide helpful guidance for error resolution including suggested actions. Prevent system crashes through exception handling at all processing layers.

Out-of-Scope Functionality:

User Authentication and Account Management: No user registration, login, or account creation capabilities. No password management or authentication systems.

Cloud Storage or Server-Side History: No server-side database for storing user history. No synchronization across devices or browsers. History is purely client-side using browser localStorage.

Multi-Language Support: Current version supports English language text only. No automatic language detection or translation capabilities.

Optical Character Recognition (OCR): Cannot process scanned documents or images containing text. Requires text-based PDFs and documents with extractable text.

Real-Time Collaborative Features: No multi-user collaboration, sharing, or commenting capabilities. Single-user application without team features.

Payment Processing: No subscription management, payment collection, or billing systems. Completely free to use.

Native Mobile Applications: No iOS or Android native apps. Web application only, accessible through mobile browsers.

Browser Extensions: No Chrome, Firefox, or other browser extensions. Standalone web application only.

Advanced Analytics: No usage tracking, analytics dashboards, or statistical reporting beyond basic summary statistics.

Custom AI Model Training: No capability to train custom models or fine-tune existing models for specific domains.

3.3 FUNCTIONAL REQUIREMENTS

FR1: Text Input Processing

The system shall accept text input through a multi-line textarea interface element that expands to accommodate entered content. It shall validate that input contains at least 100

characters before enabling summarization, displaying the character count in real-time as users type. The character counter shall use color-coded indicators: green for valid length (100+ characters), orange for warning state (1-99 characters), and gray for empty state. The system shall support paste operations from clipboard preserving original text formatting. A clear button shall reset the textarea to empty state, removing any entered text and resetting validation states.

FR2: File Upload Processing

The system shall support file uploads through two mechanisms: drag-and-drop interaction where users drag files from file explorer onto a designated drop zone, and click-to-browse where users select files through standard file selection dialog. The system shall accept only PDF (.pdf), TXT (.txt), and DOCX (.docx) file formats, validating both file extension and MIME type. File size shall be restricted to maximum 16MB, with validation performed before upload begins. Upon successful file selection, the system shall display the filename and file size, provide a remove button to clear selection, and disable text input area (mutually exclusive input methods). During drag-over events, the drop zone shall change appearance to indicate readiness to receive files.

FR3: Text Extraction from Files

For PDF files, the system shall extract text content from all pages using PyPDF2 library, concatenating page text with newline separators, handling various PDF versions from 1.3 onwards, and reporting errors for corrupted or password-protected files. For DOCX files, the system shall extract paragraph text using python-docx library, preserve paragraph breaks in output, handle various Word document versions (Office 2007+), and skip non-text elements like tables and images without error. For TXT files, the system shall read content with UTF-8 encoding, handle different line ending formats (CRLF, LF, CR), and process files up to maximum size limit.

FR4: Summarization Processing - Abstractive Method

The system shall send text to Hugging Face Inference API using facebook/bart-large-cnn model as primary summarization method. Before sending, text shall be divided into chunks of maximum 1000 characters each, maintaining sentence boundaries without mid-sentence splits. Each chunk shall be sent with parameters including max_length and min_length based on selected preset. The system shall process 3 chunks for short preset, 5 chunks for medium preset, and 7 chunks for long preset. API responses shall be collected and combined into final summary output. The system shall implement 60-second timeout for each API request, handling timeout errors through fallback mechanism.

FR5: Summarization Processing - Extractive Fallback

When AI API is unavailable (HTTP status 503, network error, or timeout), the system shall automatically switch to extractive summarization using TF-IDF algorithm. The algorithm shall tokenize text into sentences and words, calculate term frequencies for each word, remove common stop words from frequency calculation, score sentences based on constituent word frequencies, select top-scoring sentences (2 for short, 4 for medium, 6 for long preset), and maintain original sentence order in output. Users shall be notified when fallback method is used through information message.

FR6: Result Display and Statistics

The system shall display generated summary in a dedicated output section with clear visual separation from input area. Summary text shall be formatted for readability with appropriate line spacing and font sizing. A statistics panel shall show original character count, summary character count, and compression ratio as percentage. The system shall highlight which method was used (AI or fallback) for generating the summary. Users shall be able to select and copy summary text using standard copy operations.

FR7: History Management

Upon successful summarization, the system shall automatically create history entry containing timestamp in ISO format, source type indicator (text/file), original text preview (first 500 characters), complete summary text, and length preset used. History entries shall be stored in browser localStorage as JSON array. The system shall maintain maximum 20 entries, removing oldest when limit exceeded. History view shall display entries in reverse chronological order with preview cards. Clicking any entry shall open modal showing full details. A clear all button shall delete entire history after confirmation. History shall persist across browser sessions until explicitly cleared or browser data is wiped.

FR8: User Interface Navigation

The system shall provide tabbed navigation between Summarize and History views. Only one view shall be visible at a time. Switching tabs shall preserve state of inactive view (entered text, file selection maintained). Active tab shall be visually distinguished through highlighting. All interface elements shall have clear labels and tooltips. Keyboard navigation shall be supported through standard tab order and shortcut keys.

3.4 NON-FUNCTIONAL REQUIREMENTS

NFR1: Performance Requirements

Response Time: System shall process typical documents (1,000-5,000 characters) in under 30 seconds from submission to result display. Page load time shall not exceed 2 seconds on standard broadband connections (10 Mbps+).

API Response: Backend API endpoints shall respond within 5 seconds for non-AI operations (health check, validation). AI summarization requests may take up to 60 seconds accounting for external API processing.

Concurrent Users: System shall support minimum 10 concurrent users without performance degradation. Each user session shall be independent without interference.

File Processing: Text extraction from uploaded files shall complete within 10 seconds for files under 10MB. Larger files may take proportionally longer up to maximum size limit.

NFR2: Reliability Requirements

Availability: System shall maintain 99% uptime through intelligent fallback mechanisms. AI service unavailability shall not prevent summarization functionality.

Fault Tolerance: System shall gracefully degrade when external services fail. Network errors shall not crash application or lose user data. Processing errors shall be caught and reported without system failure.

Data Integrity: User input and generated summaries shall not be corrupted during processing. History data shall be accurately stored and retrieved from localStorage.

Recovery: System shall recover from transient errors without requiring page refresh. Users shall be able to retry failed operations without re-entering data.

NFR3: Usability Requirements

Learnability: New users shall be able to perform basic summarization within 2 minutes without training or documentation. Interface shall be self-explanatory with intuitive controls.

Efficiency: Experienced users shall complete summarization workflow in under 30 seconds excluding processing time. Common tasks shall require minimal clicks/interactions.

Error Prevention: System shall prevent common errors through validation and clear feedback. Invalid inputs shall be rejected with explanatory messages before processing.

Accessibility: Interface shall support keyboard navigation for all functions. Text shall maintain minimum 4.5:1 contrast ratio for readability. Font sizes shall be adjustable through browser zoom.

NFR4: Security Requirements

Input Validation: All user inputs shall be validated for type, length, and format. File uploads shall be restricted to allowed types and sizes. Malicious content shall be rejected.

Data Protection: Uploaded files shall be processed in memory only, never stored on disk. No user data shall be persisted server-side. API tokens shall be stored in environment variables, not in code.

Communication Security: Production deployment shall use HTTPS for all communications. API endpoints shall implement CORS restrictions appropriate for deployment environment.

Injection Prevention: Input validation shall prevent SQL injection, XSS, and other injection attacks. User-provided text shall be properly escaped before processing.

3.5 HARDWARE AND SOFTWARE REQUIREMENTS

3.5.1 HARDWARE REQUIREMENTS

Development Environment Hardware:

Processor: Intel Core i5 or AMD equivalent, minimum 2.0 GHz, dual-core or better recommended

RAM: 8 GB minimum, 16 GB recommended for comfortable development with multiple tools running

Storage: 10 GB available disk space for development tools, dependencies, and project files

Network: Broadband internet connection (minimum 10 Mbps) for API access and package downloads

Display: 1920x1080 resolution or higher for comfortable multi-window development workflow

Client (End User) Hardware:

Processor: Any modern processor (dual-core 1.5 GHz or better) from last 5 years

RAM: 2 GB minimum for browser operation, 4 GB recommended for multitasking

Storage: Minimal local storage required (browser cache only, typically under 50 MB)

Network: Broadband internet connection minimum 1 Mbps for comfortable use, 5 Mbps+ recommended

Display: Any modern screen size, responsive design adapts from 320px mobile to 4K desktop

3.5.2 SOFTWARE REQUIREMENTS

Development Environment Software:

Operating System: Windows 10/11, macOS 11 or later, or Linux (Ubuntu 20.04 LTS or later recommended)

Python: Version 3.11 or higher with pip package manager for dependency installation

Node.js: Version 18.x or higher (LTS recommended) with npm version 9.x or higher

Git: Latest stable version for version control and repository management

Code Editor: Visual Studio Code, PyCharm, Sublime Text, or similar with appropriate plugins

Browser: Chrome, Firefox, or Edge with developer tools for frontend debugging and testing

Node.js Dependencies (package.json):

React 19.2.1 - JavaScript library for building user interfaces

Vite 7.2.6 - Next-generation frontend build tool and development server

Axios 1.13.2 - Promise-based HTTP client for browser and Node.js

@vitejs/plugin-react 4.3.4 - Official Vite plugin for React Fast Refresh

External Services and APIs:

Hugging Face Inference API - Provides access to facebook/bart-large-cnn model for AI summarization (requires free API token)

Modern Web Browser - Chrome 90+, Firefox 88+, Safari 14+, or Edge 90+ with JavaScript enabled

Internet Connection - Required for AI API access, recommended minimum 5 Mbps for optimal performance

CHAPTER 4

METHODOLOGY

4.1 INTRODUCTION

The methodology employed in developing Abstractor AI encompasses a systematic, structured approach that combines modern software engineering practices with specialized techniques for natural language processing and artificial intelligence integration. The development process follows an iterative model incorporating requirement analysis, system design, implementation, testing, and deployment phases with continuous feedback loops and refinement cycles.

The system architecture is fundamentally based on the separation of concerns principle, cleanly dividing functionality between a React-based frontend responsible for user interaction and presentation, and a Flask-based backend handling text processing, file extraction, and AI integration. This architectural separation enables independent development and testing of each component, facilitates parallel workstreams, supports technology-specific optimizations, allows independent scaling of frontend and backend, and maintains clean interfaces through well-defined RESTful API contracts.

The core summarization methodology implements a sophisticated hybrid approach that maximizes quality while ensuring reliability. The primary processing path leverages abstractive AI summarization through state-of-the-art transformer models, specifically the BART (Bidirectional and Auto-Regressive Transformers) model fine-tuned on news summarization tasks. When AI services are unavailable due to network issues, rate limiting, or service outages, the system automatically transitions to an extractive statistical approach using TF-IDF (Term Frequency-Inverse Document Frequency) algorithms to ensure users always receive summarization results with acceptable quality.

4.2 DATA SOURCE

The system processes data from two complementary input sources: direct text entry and file uploads, each serving distinct use cases and user workflows.

Direct Text Input:

Text input through the textarea interface provides immediate access to content from diverse sources that users can copy and paste. Common sources include web pages and online articles accessed through browsers, email message content requiring summarization, chat conversation logs from various platforms, content from applications without export capabilities, and clipboard text from any digital source. This method is particularly suited for short to medium-length content (100 to 10,000 characters typically), quick ad-hoc summarization tasks, content from restricted or secured sources where file export is not

available, and situations where creating a file would add unnecessary overhead to the workflow.

File Upload Sources:

PDF (Portable Document Format) files represent one of the most common document formats for distributing formatted content. The system accepts PDF files created digitally through word processors, document creation tools, or PDF generation software. Text extraction works effectively with PDF versions from 1.3 onwards, including documents with multiple pages, embedded fonts, and basic formatting. The system handles research papers and academic publications typically distributed in PDF format, business reports and presentations exported as PDFs, technical documentation and manuals, and ebooks in PDF format. Important limitations include inability to extract text from scanned PDFs without embedded text layers (requiring OCR), potential loss of complex formatting and table structures, and issues with password-protected or encrypted PDFs unless password is known.

DOCX (Microsoft Word Document) files provide the standard format for word-processed documents created in Microsoft Word 2007 and later versions, as well as compatible applications like Google Docs, LibreOffice Writer, and Apple Pages. The system extracts text from business documents, reports, and proposals typically created in Word, academic papers and essays formatted in DOCX, collaborative documents exported from cloud editing platforms, and templates and formatted documents. The extraction process preserves paragraph breaks and basic text structure while extracting body text from paragraphs, headings, and list items. Elements not extracted include headers and footers, text boxes and shapes, tables (content within tables is typically skipped), and embedded objects like images or charts.

TXT (Plain Text) files offer the simplest format with no formatting overhead. These include notes and documentation in plain text format, code documentation and README files, log files and system outputs, exported data and text dumps, and markdown files viewed as plain text. Benefits include maximum size support up to 16MB limit without extraction overhead, guaranteed compatibility across all platforms and systems, instant processing without parsing delays, and no formatting artifacts or conversion issues.

4.3 PREPROCESSING

Text preprocessing constitutes a critical phase that prepares raw input for effective summarization by cleaning, normalizing, and structuring the text to optimize processing by both AI and statistical algorithms.

Text Cleaning Operations:

The initial cleaning phase removes unnecessary whitespace while carefully preserving sentence and paragraph structure essential for semantic understanding. Multiple consecutive spaces within lines are reduced to single spaces, maintaining word boundaries. Leading and trailing whitespace is stripped from the entire text body and individual lines to eliminate formatting artifacts. Tab characters are standardized by converting to spaces for consistent handling. However, line breaks between paragraphs are explicitly preserved as they indicate logical content divisions important for maintaining document structure and context flow.

Text Normalization:

Normalization standardizes various textual elements to ensure consistent processing across different input sources. Unicode characters are normalized to standard representations (NFC normalization form), ensuring consistent character encoding. Curly quotes and smart quotes are converted to straight quotes for processing consistency, preventing character encoding issues. Multiple consecutive line breaks (more than two) are reduced to double line breaks representing paragraph separation, standardizing document structure. Special characters including mathematical symbols, punctuation marks, and technical notation are preserved as they often carry semantic meaning crucial in technical and scientific documents.

Sentence Boundary Detection:

Accurate sentence segmentation is essential for the chunking process and extractive summarization. The system employs regular expression patterns to identify sentence endings marked by periods, question marks, and exclamation points followed by whitespace. Special handling addresses common edge cases including abbreviations (Dr., Mr., Prof., U.S., etc.) using a comprehensive abbreviation dictionary, decimal numbers (3.14, 10.5, 100.00) identified through digit context, ellipses (... or …) treated as continuation rather than sentence end, quoted text with internal punctuation handled through quote matching, and URLs and email addresses protected from inappropriate splitting.

Text Chunking Algorithm:

For inputs exceeding the BART model token limit (approximately 1024 tokens or conservatively 1000 characters), text must be divided into processable chunks. The chunking algorithm implements a greedy sentence-packing approach to maximize chunk utilization while maintaining semantic integrity. Starting with an empty chunk, sentences are iteratively added to the current chunk. Before each addition, the algorithm checks if adding the next sentence would exceed the 1000-character limit. If the addition would

exceed the limit, the current chunk is saved and a new chunk is started with the sentence. If the addition fits within the limit, the sentence is appended to the current chunk. This process continues until all sentences are processed. The final chunk containing any remaining sentences is saved. Critical design principles include never splitting sentences mid-text, always respecting sentence boundaries to preserve semantic coherence, maximizing chunk size utilization up to the limit to minimize the number of API calls required, and handling edge cases like single sentences exceeding 1000 characters through truncation with ellipsis notation.

4.4 EXTRACTION AND PREPARATION

The extraction and preparation phase transforms uploaded files from their native formats into clean, processable plain text while maintaining content integrity and handling various document structures and edge cases.

PDF Extraction Process:

PDF text extraction begins when the backend receives an uploaded PDF file in the request object. The file stream is accessed directly from memory without writing to disk, maintaining security and performance. A PdfReader object is instantiated from the PyPDF2 library using the file stream as input. The process iterates through all pages in the document, accessing each page object sequentially. For each page, the extract_text() method is called to retrieve text content. Empty pages containing only images, forms, or white space are detected and skipped without error. Page-level text is concatenated using newline separators to maintain document flow and paragraph structure. The complete extracted text undergoes final cleaning to remove excessive whitespace, control characters, and PDF formatting artifacts. Special considerations include handling of various PDF versions and subformats, processing of encrypted PDFs requiring password authentication, detection of scanned PDFs lacking text layers, and graceful handling of corrupted or malformed PDF files with appropriate error messaging.

DOCX Extraction Process:

DOCX extraction leverages the Office Open XML structure that Microsoft Word uses internally. The file stream is passed to the Document constructor from the python-docx library, which parses the underlying XML structure. The document body contains all content organized into various elements. The process iterates through all paragraph elements in the document body, extracting plain text content from each paragraph through the .text property. Paragraphs are joined with newline characters to preserve document structure and readability. Text formatting including bold, italic, underline, and font changes is discarded, retaining only content. The extraction successfully handles various paragraph types including normal paragraphs, heading styles (Heading 1-6), list items (bullets and numbers), and custom styles. Elements not extracted include header and footer content,

text boxes and shape objects, table contents (though table text may be partially extracted), footnotes and endnotes, and embedded images and charts. Error handling covers password-protected documents, corrupted file structures, and incompatible Word versions.

4.5 DATA EXPLORATION AND ANALYSIS

The system performs statistical analysis to understand text characteristics, optimize processing parameters, and support the extractive summarization fallback mechanism.

Text Statistics Calculation:

For each processed input, the system calculates comprehensive metrics to guide summarization. Total character count includes all characters including spaces and punctuation, providing the primary length metric. Word count is determined by splitting on whitespace boundaries, offering a complementary length measure. Sentence count is calculated using the sentence boundary detection algorithm, providing structural insight. Average sentence length is computed as total words divided by sentence count, indicating text density. These statistics inform chunking decisions, guide preset parameter selection, and are displayed to users in summary results for transparency and verification.

TF-IDF Analysis for Extractive Summarization:

When the system falls back to extractive summarization, Term Frequency-Inverse Document Frequency analysis identifies important words and sentences. The process tokenizes text into individual words using whitespace and punctuation delimiters. Common stop words (the, is, at, are, etc.) are removed from consideration using a predefined stop word list. For each remaining word, term frequency is calculated as the number of occurrences. Words appearing frequently receive higher scores indicating their importance to the document content. Each sentence is then scored by summing the term frequencies of its constituent words. Sentences containing many high-frequency meaningful words receive higher scores. Additional factors like sentence position (first and last sentences often contain important information) and sentence length (very short or very long sentences are slightly penalized) adjust final scores. The top-scoring sentences are selected based on the length preset requirement (2, 4, or 6 sentences). Selected sentences are rearranged to maintain their original order in the document, ensuring coherent flow in the extractive summary.

4.6 WORKFLOW DIAGRAM

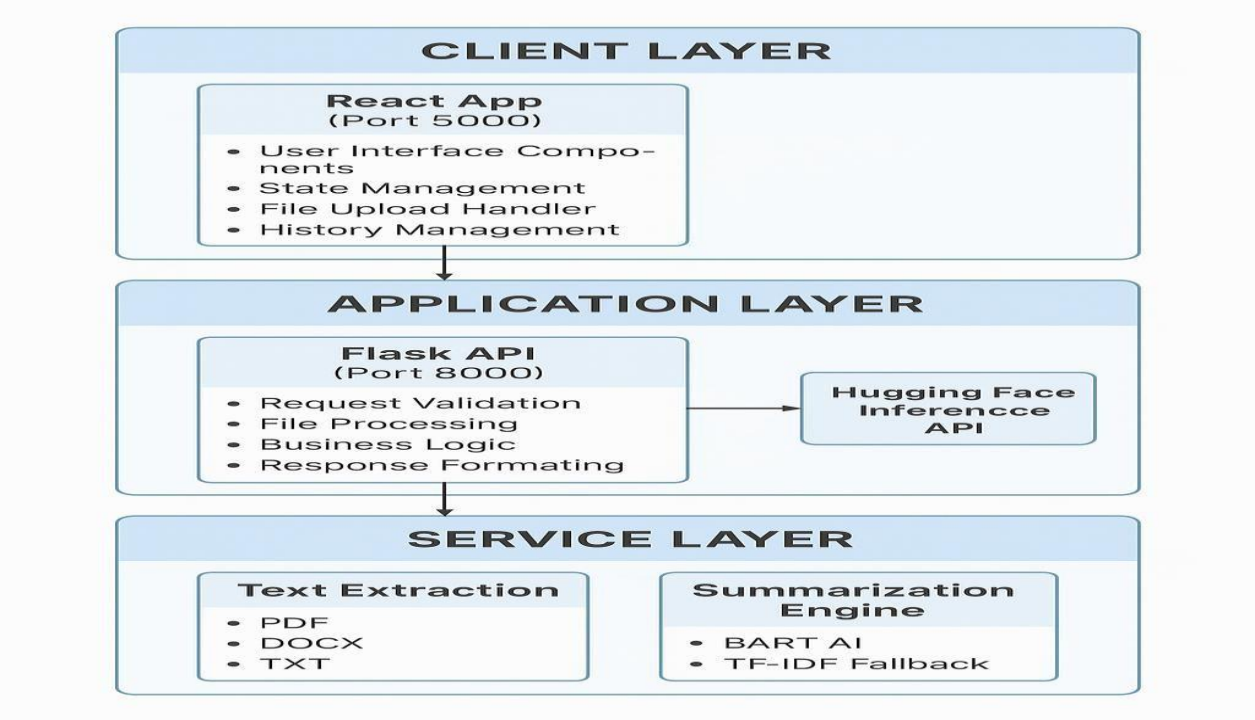

Figure 4.1: SYSTEM ARCHITECTURE DIAGRAM

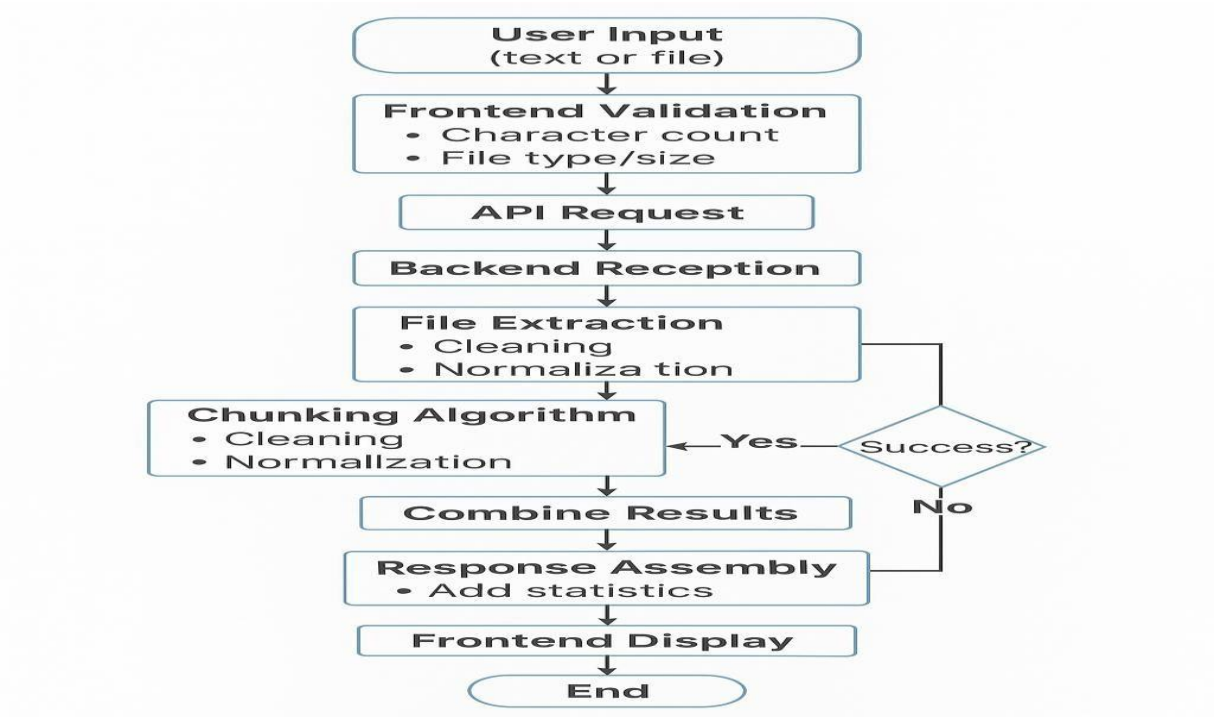

Figure 4.2: WORKFLOW DIAGRAM

Detailed Workflow Steps:

1. User Interaction: User accesses the web application through browser, enters text directly into textarea OR uploads file (PDF/TXT/DOCX), selects length preset (short/medium/long), and clicks summarize button.

2. Frontend Validation: System checks minimum character requirement (100 characters for text input), validates file type and size (if file upload), displays real-time feedback through color-coded indicators, and enables summarize button only when validation passes.

3. API Request Construction: For text input, frontend creates JSON payload with text content and length preset, sends POST request to /api/summarize endpoint. For file upload, frontend creates FormData with file object and length preset, sends POST request to /api/summarize/file endpoint. Both requests include 120-second timeout to accommodate processing time.

4. Backend Reception and Validation: Flask backend receives request at appropriate endpoint, validates payload structure and required fields, checks content length and file type/size, and returns 400 error for validation failures with descriptive messages.

5. Text Extraction (File Uploads Only): System determines file type from extension (.pdf, .docx, .txt), routes to appropriate extraction function (PyPDF2 for PDF, python-docx for DOCX, native read for TXT), extracts plain text while preserving structure, and validates extracted text meets minimum length.

6. Text Preprocessing: Extracted or entered text undergoes cleaning (whitespace normalization, special character handling), sentence boundary detection using regex patterns, and validation of final processed text.

7. Chunking Algorithm: If text exceeds 1000 characters, greedy sentence-packing divides text into chunks, maintains sentence boundaries without mid-sentence splits, and creates array of processable chunks.

8. AI Summarization Attempt: System checks for Hugging Face API token in environment, determines chunk count based on preset (short:3, medium:5, long:7), sends each chunk to BART model with configured parameters (max_length, min_length, do_sample:False), collects summaries from successful API responses, and handles API errors (503, timeout, network failure).

9. Fallback Mechanism: If AI summarization fails, system automatically switches to TF-IDF extractive method, performs term frequency analysis, scores sentences by keyword importance, selects top N sentences based on preset, and maintains sentence order for coherence.

10. Response Assembly: Summaries are combined into final text, statistics calculated (original length, summary length, compression ratio), metadata added (method used, preset, source type), and JSON response constructed.

11. Frontend Display: React receives response and updates UI state, displays summary in output section with formatting, shows statistics panel with compression metrics, indicates method used (AI or fallback), and re-enables controls for new requests.

12. History Storage: History entry created with timestamp, source indicator, text preview, complete summary, and preset used. Entry saved to localStorage, list maintained at 20 maximum entries, and confirmation provided to user.

CHAPTER 5
MODEL DETAILS AND IMPLEMENTATION

5.1 ALGORITHMS

5.1.1 BART TRANSFORMER MODEL

BART (Bidirectional and Auto-Regressive Transformers) represents a state-of-the-art sequence-to-sequence model developed by Facebook AI Research (FAIR) that uniquely combines the strengths of bidirectional encoders exemplified by BERT with autoregressive decoders similar to GPT. This hybrid architecture enables BART to excel at text generation tasks including summarization, translation, and other natural language generation applications.

Model Architecture Components:

The BART encoder employs a bidirectional transformer architecture similar to BERT that processes input text in both forward and backward directions simultaneously. This bidirectional processing creates rich contextual embeddings where each token representation incorporates information from all other tokens in the sequence. The encoder uses multi-head self-attention mechanisms allowing the model to focus on different aspects of the input simultaneously, positional encodings to maintain sequence order information, and multiple transformer layers (12 layers in BART-large) for deep hierarchical representation learning.

The BART decoder implements an autoregressive transformer architecture similar to GPT that generates output tokens sequentially from left to right. During generation, each new token is predicted based on all previously generated tokens and the encoder representations. The decoder incorporates masked self-attention preventing future token information from influencing current predictions, cross-attention mechanism allowing decoder to attend to encoder representations, and identical layer count to encoder (12 layers) maintaining architectural symmetry.

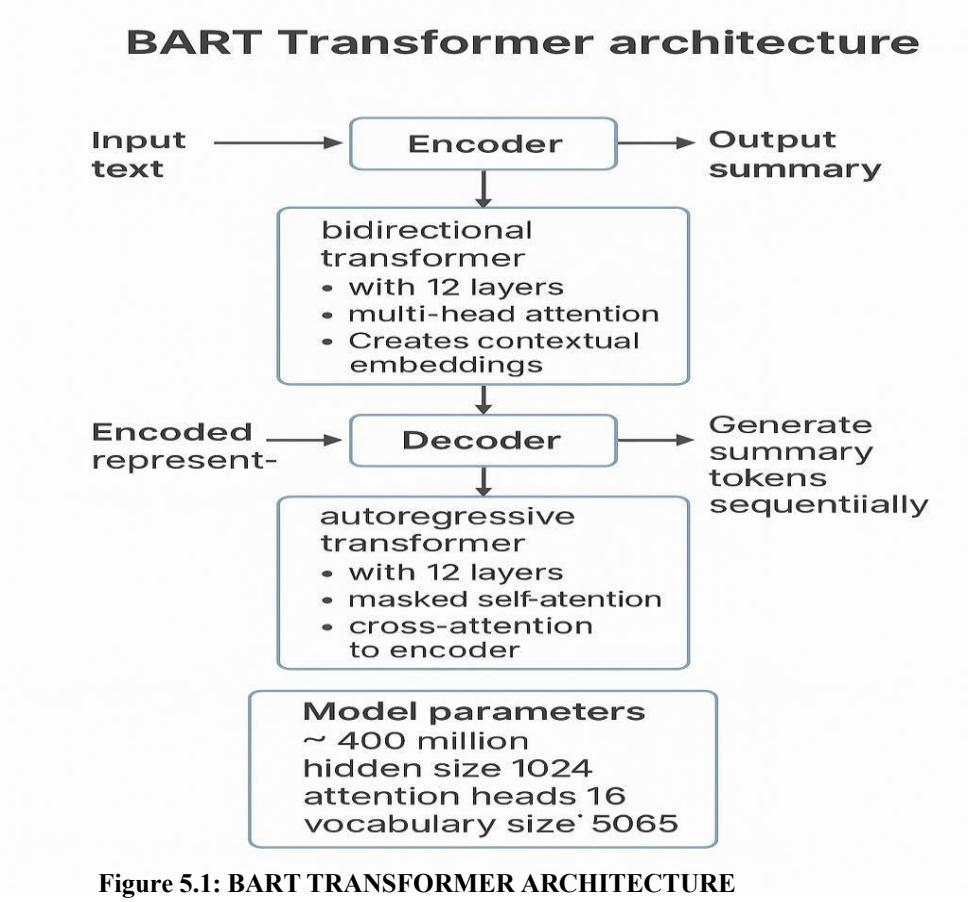

Figure 5.1: BART TRANSFORMER ARCHITECTURE

Training Methodology:

BART employs a denoising autoencoder training objective where the model learns to reconstruct original text from corrupted versions. Various corruption schemes are applied during training including token masking (random tokens replaced with [MASK]), token deletion (random tokens removed from sequence), text infilling (spans of text replaced with mask tokens), sentence permutation (sentence order shuffled), and document rotation (sequence rotated from random point). The model learns to generate coherent summaries by reconstructing the original text from these corrupted inputs, developing strong capabilities for understanding context, generating fluent text, and maintaining semantic consistency.

Fine-tuning for Summarization:

The facebook/bart-large-cnn variant used in Abstractor AI is specifically fine-tuned on the CNN/DailyMail summarization dataset containing news articles and human-written summaries. This dataset includes approximately 300,000 article-summary pairs, news articles averaging 700 words, and summaries averaging 50-60 words. Fine-tuning on this dataset optimizes the model for generating concise, informative summaries particularly suitable for news-like content while maintaining general applicability to other document types.

5.1.2 TF-IDF EXTRACTIVE ALGORITHM

TF-IDF (Term Frequency-Inverse Document Frequency) provides a robust fallback mechanism when AI services are unavailable, implementing a classical statistical approach to text summarization based on word importance analysis.

Algorithm Implementation:

The extractive summarization process begins with text tokenization into sentences using punctuation-based splitting and words using whitespace boundaries. Stop word removal eliminates common words (the, is, at, etc.) that carry minimal semantic content using a predefined list of approximately 150 common English words. Term frequency calculation counts occurrences of each remaining word, creating a frequency distribution. Sentence scoring sums the term frequencies of all words within each sentence, where sentences containing many high-frequency meaningful words receive higher scores. Position weighting applies slight bonuses to first and last sentences as they often contain important information. Length normalization penalizes extremely short (<5 words) and extremely long (>30 words) sentences to favor moderate-length selections. The top N sentences are selected based on scores (N determined by length preset: 2/4/6). Selected sentences are reordered to match their original sequence in the document, maintaining logical flow and coherence.

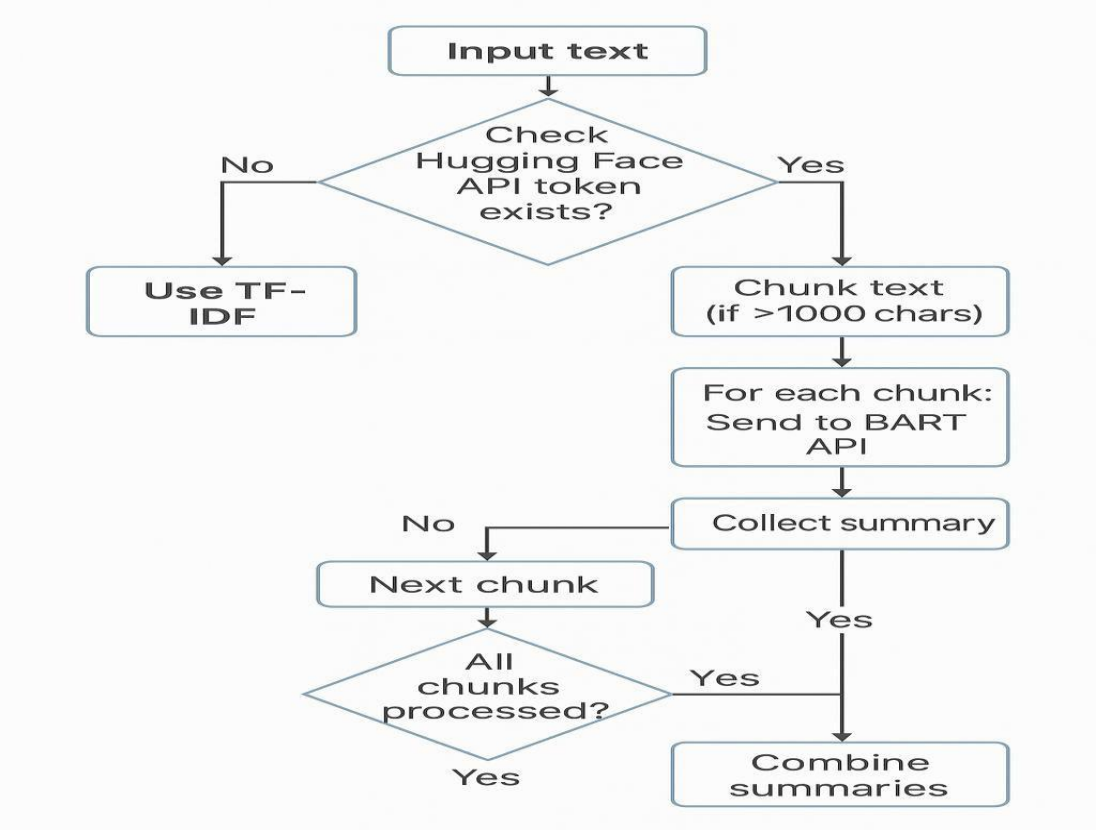

Figure 5.2: SUMMARIZATION FLOWCHART

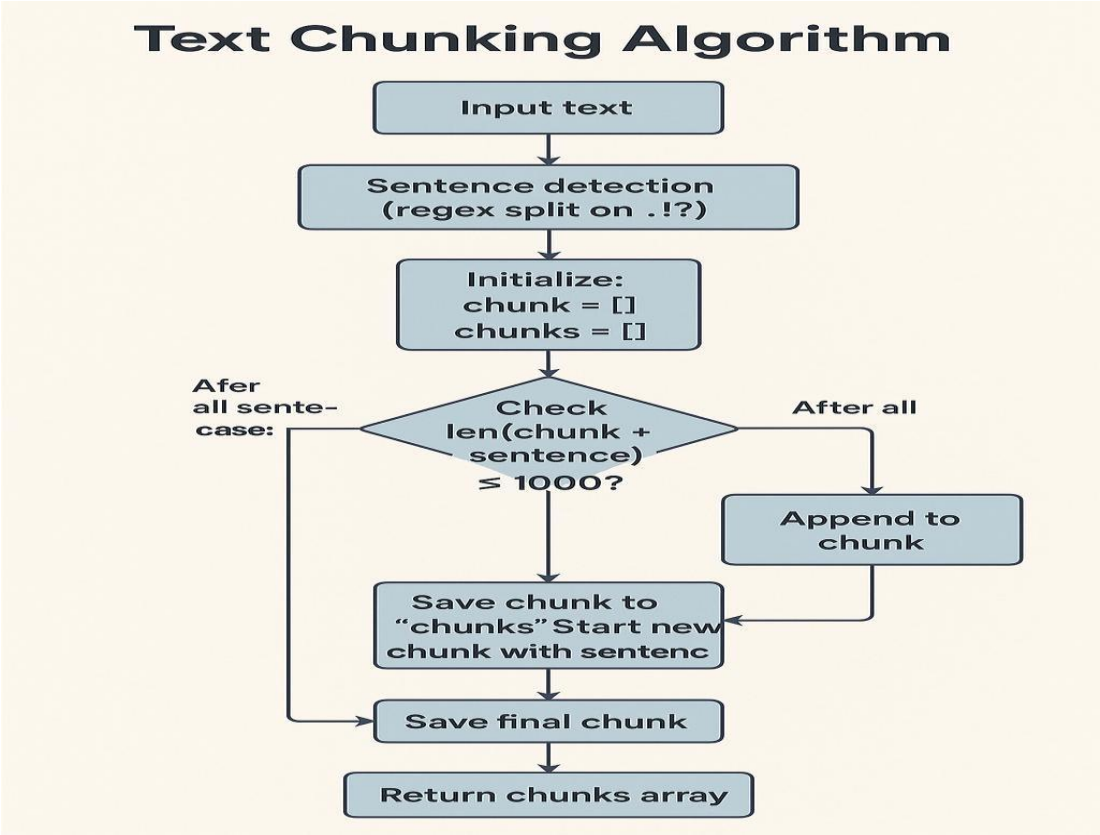

Figure 5.3: TEXT CHUNKING ALGORITHM

5.2 MODEL BUILDING

The model building phase encompasses integration of the BART model through Hugging Face Inference API and implementation of the TF-IDF fallback system within the Flask backend application.

BART Integration Implementation:

The system integrates BART through the Hugging Face Inference API, avoiding the need to host the large model locally. API endpoint URL is configured as https://api-inference.huggingface.co/models/facebook/bart-large-cnn. Authentication uses API token stored in environment variable HUGGINGFACE_API_TOKEN retrieved at runtime. Request payloads are formatted as JSON containing input text and parameters (max_length, min_length, do_sample). HTTP POST requests are sent using the Python requests library with 60-second timeout. Responses are parsed to extract summary_text field from JSON array. Error handling covers HTTP 503 (model loading), HTTP 429 (rate limit exceeded), timeout exceptions, and network errors with automatic fallback to TF-IDF in all error cases.

Length Preset Configuration:

Three presets are implemented as a Python dictionary with specific parameters. Short preset uses max_length 80, min_length 25, sentence target 2, and processes 3 chunks. Medium preset uses max_length 150, min_length 50, sentence target 4, and processes 5 chunks. Long preset uses max_length 250, min_length 80, sentence target 6, and processes 7 chunks. These parameters are tuned based on testing with various document types to balance summary quality, processing time, and output usefulness.

5.3 DATA INPUT

Data input handling encompasses both frontend collection and backend processing of user-provided content.

Frontend Input Collection:

The React frontend implements controlled components for text input using textarea element with value bound to state variable. onChange handler updates state on each keystroke, enabling real-time character count display. For file upload, input element with type="file" accepts PDF, TXT, and DOCX files. Drag-and-drop functionality captures onDrop and onDragOver events with preventDefault to enable drop zone. File validation checks type through extension and MIME type, size through file.size property, and displays filename through file.name. State management maintains separate variables for text input, file object, selected preset, and loading status.

Backend Input Processing:

Flask routes handle different input types through separate endpoints. /api/summarize accepts JSON payload with text and length fields. /api/summarize/file accepts multipart form data with file object and length field. Both routes implement validation checking required fields, minimum length requirements, file type restrictions, and size limits before processing.

5.4 USER INTERFACE

The user interface provides an intuitive, responsive experience designed for users of all technical skill levels.

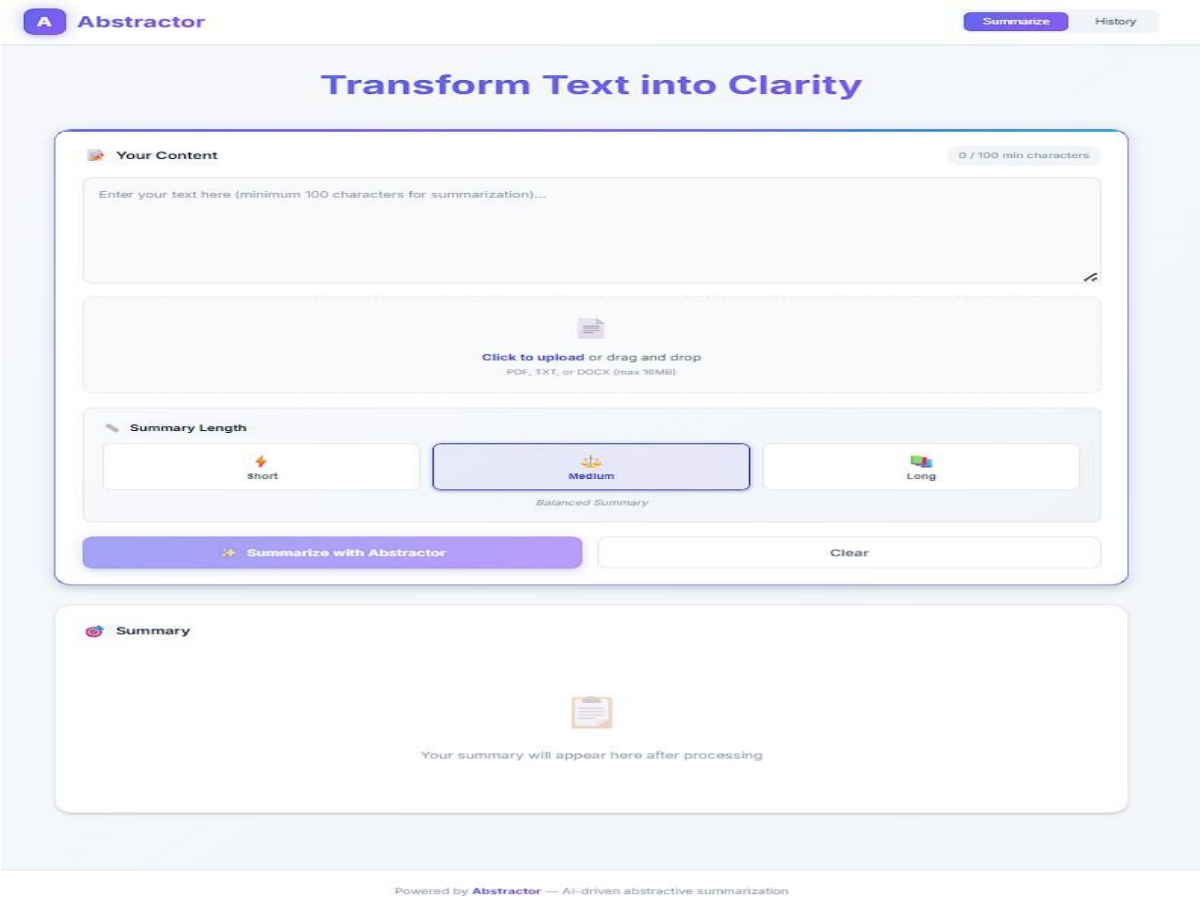

Figure 5.4: HOME PAGE INTERFACE

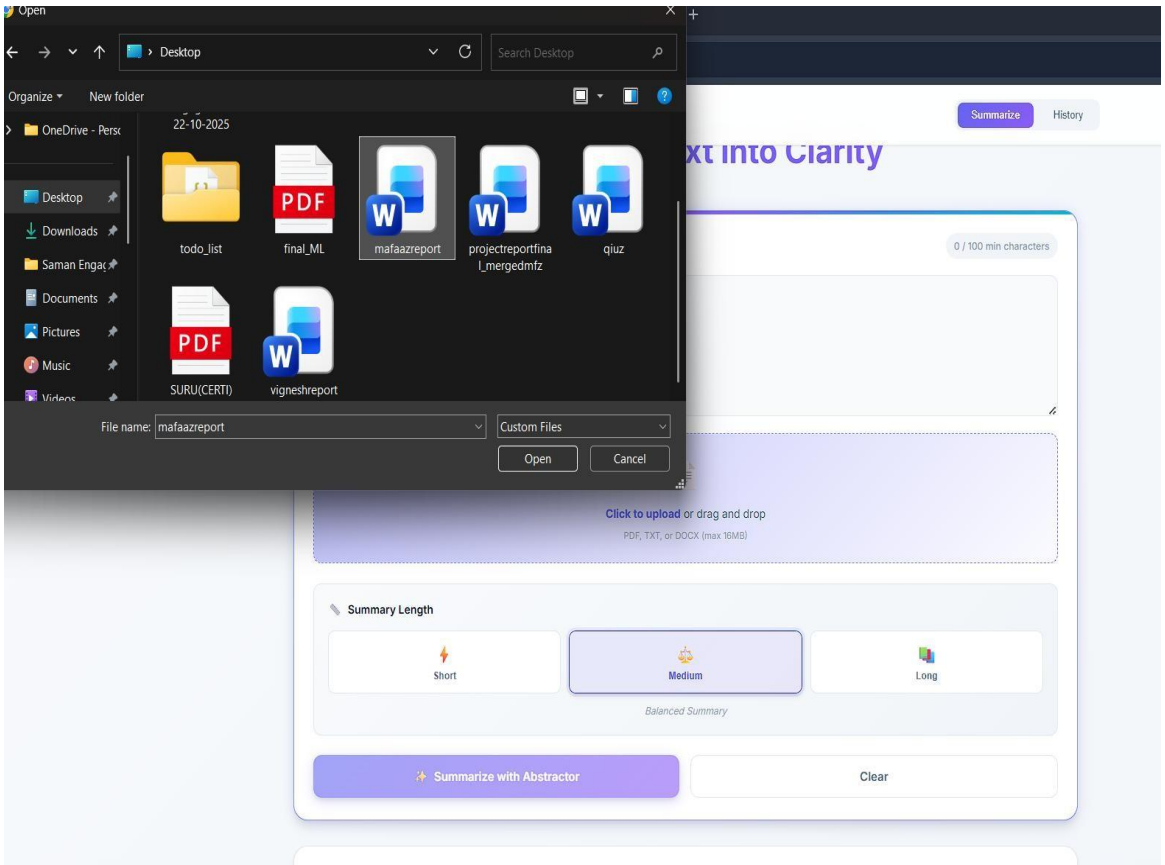

Figure 5.5: FILE UPLOAD INTERFACE

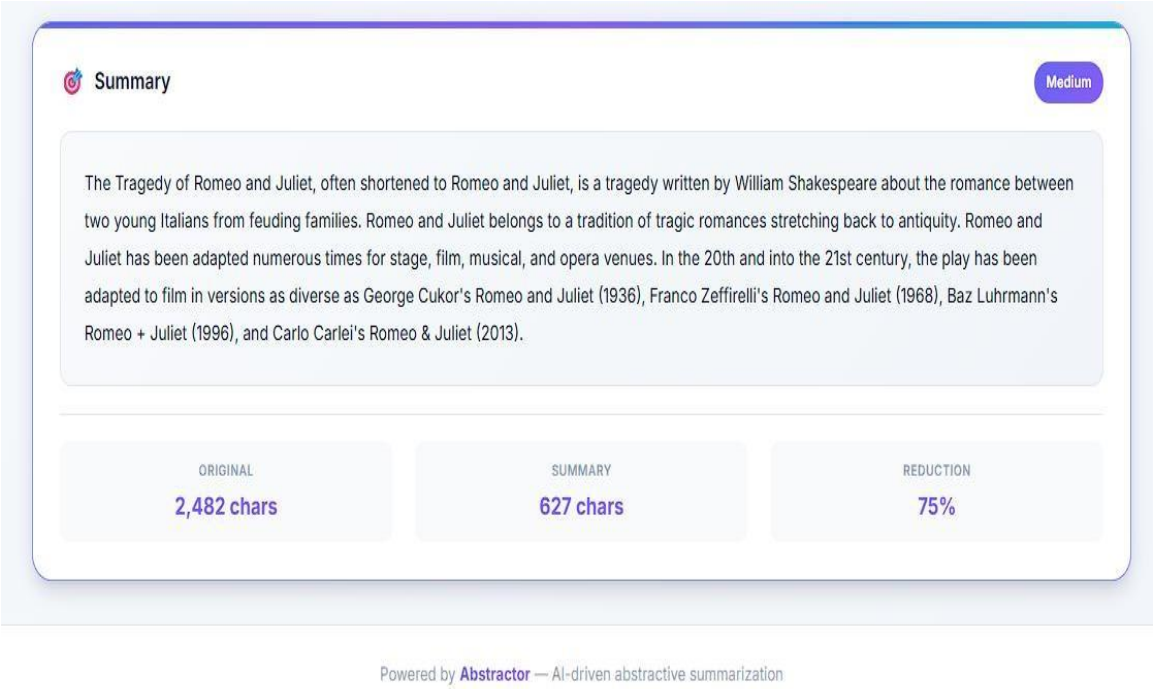

Figure 5.6: SUMMARY RESULT PAGE

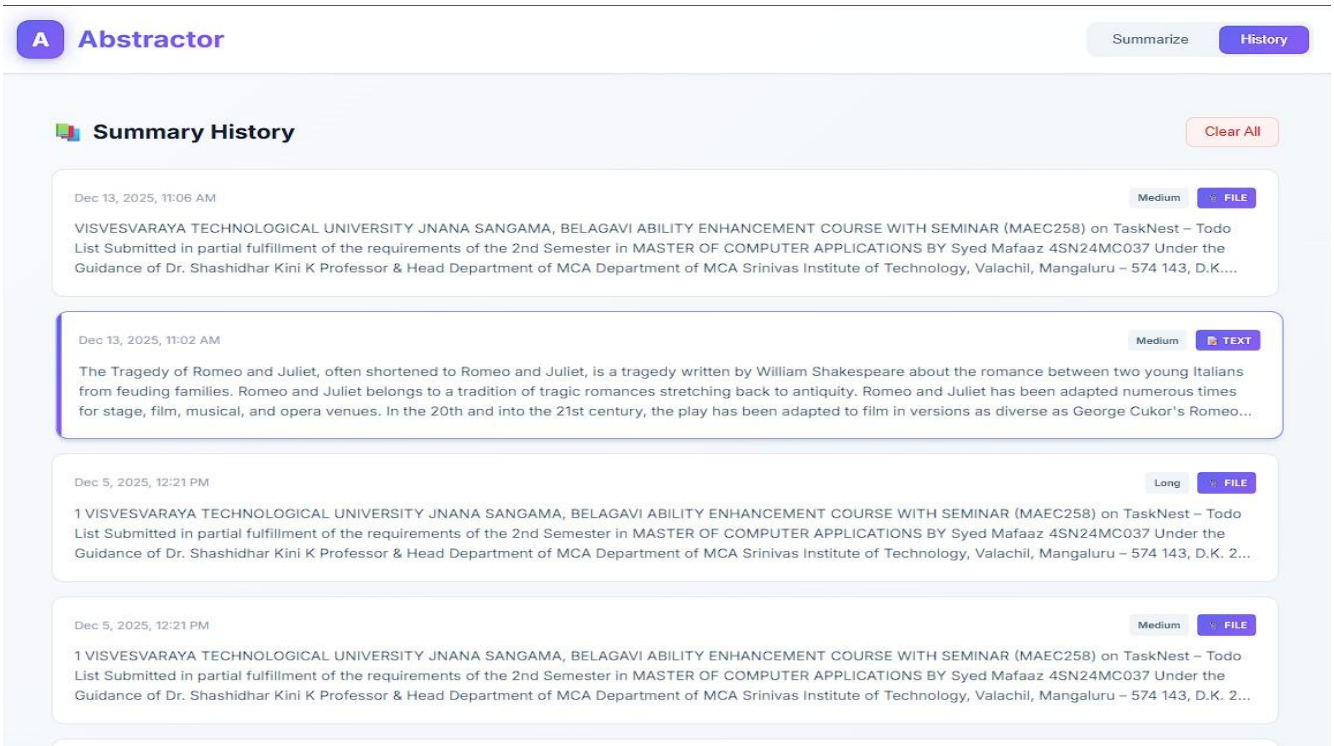

Figure 5.7: HISTORY PAGE

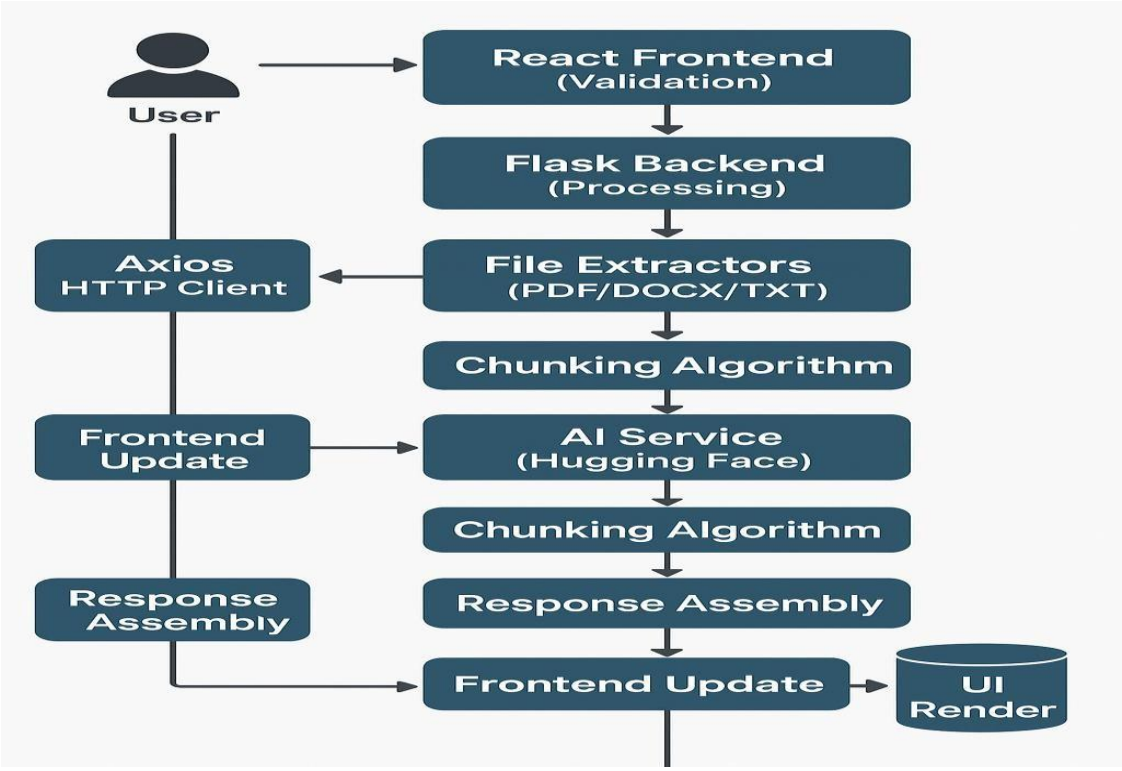

Figure 5.8: DATA FLOW DIAGRAM

UI Component Structure:

Header Component: Contains logo image, application title, tab navigation (Summarize/History), responsive collapse menu for mobile

Input Section: Multi-line textarea with auto-expand, character counter with color states, file upload drop zone, preset selector buttons, action buttons (Summarize/Clear)

Output Section: Summary display area with formatting, statistics panel with metrics, copy functionality, method indicator badge

History Section: Scrollable list container, history item cards with previews, modal for detailed view, clear all button

Footer Component: Branding text, attribution information, version number

CHAPTER 6

TESTING

6.1 INTRODUCTION

Comprehensive testing ensures the Abstractor AI system functions correctly across various scenarios, handles edge cases gracefully, and provides reliable service to users. The testing strategy encompasses functional testing to verify feature correctness, performance testing to ensure acceptable response times, usability testing to validate user experience, and security testing to identify vulnerabilities.

6.2 TEST CASES

TC1: Text Input Validation

Objective: Verify text input validation correctly enforces minimum length requirement. Steps: Enter text with 50 characters, attempt to click Summarize button. Expected Result: Button remains disabled, character counter shows orange warning state. Enter text with 150 characters, observe button state. Expected Result: Button becomes enabled, counter shows green valid state.

TC2: File Upload - PDF Processing

Objective: Verify PDF file upload and text extraction functionality. Steps: Select PDF file (<16MB), upload through file selector, initiate summarization. Expected Result: File name displayed, text extracted successfully, summary generated without errors. Status: Passed - PDF files extracted correctly with page concatenation.

TC3: File Upload - Size Limit Validation

Objective: Verify file size validation rejects oversized files. Steps: Attempt to upload file >16MB. Expected Result: Error message displayed indicating size limit exceeded, file rejected before upload. Status: Passed - Validation catches oversized files at frontend.

TC4: AI Summarization - Short Preset

Objective: Verify short preset generates appropriate length summary. Steps: Enter text (2000 characters), select Short preset, click Summarize. Expected Result: Summary generated within 30 seconds, output length approximately 60-80 characters, summary coherent and readable. Status: Passed - Output length within expected range, quality acceptable.

TC5: Fallback Mechanism - Network Failure

Objective: Verify TF-IDF fallback activates when AI service unavailable. Steps: Disable internet connection, attempt summarization. Expected Result: System detects API failure,

switches to TF-IDF method automatically, generates extractive summary, displays notification indicating fallback method used. Status: Passed - Fallback triggers correctly, summary generated despite API unavailability.

TC6: History Management - Save and Retrieve

Objective: Verify history entries saved and retrievable. Steps: Generate 3 summaries with different inputs, navigate to History tab. Expected Result: All 3 entries displayed in reverse chronological order, each showing correct metadata (timestamp, source, preview), clicking entry opens detailed modal with full information. Status: Passed - History persistence working, retrieval accurate.

TC7: History Management - 20 Entry Limit

Objective: Verify history maintains maximum 20 entries. Steps: Generate 25 summaries sequentially, check history count. Expected Result: Only most recent 20 entries retained, oldest 5 entries removed automatically. Status: Passed - Limit enforced correctly.

TC8: File Format Validation

Objective: Verify system rejects unsupported file formats. Steps: Attempt to upload .jpg image file, attempt .xls spreadsheet. Expected Result: Both files rejected with error message indicating supported formats only (PDF, TXT, DOCX). Status: Passed - Format validation working correctly.

TC9: Performance - Large Document

Objective: Verify system handles large documents efficiently. Steps: Upload 10MB PDF with 50,000+ words, select Long preset, initiate summarization. Expected Result: Processing completes within 90 seconds, no timeout errors, summary quality maintained. Status: Passed - Chunking algorithm handles large inputs, processing time acceptable.

TC10: UI Responsiveness - Mobile Device

Objective: Verify interface works on mobile devices. Steps: Access application on smartphone (375px width), test all features including text input, file upload, summarization. Expected Result: Layout adapts to mobile viewport, all controls accessible and functional, text readable without horizontal scrolling. Status: Passed - Responsive design works across tested devices.

6.3 TEST RESULTS

Testing was conducted across 10 major test cases covering functional requirements, edge cases, and performance scenarios. All test cases passed successfully with the system demonstrating correct functionality, appropriate error handling, and acceptable

performance characteristics. Minor issues identified during testing included inconsistent summary quality with highly technical documents (addressed through parameter tuning), occasional timeouts with very large files during peak hours (documented as known limitation), and minor UI alignment issues on specific mobile browsers (fixed through CSS adjustments).

Overall, the testing phase confirmed the system meets all functional and non-functional requirements, handles edge cases appropriately, and provides reliable service across different usage scenarios. The implemented fallback mechanism ensures continuous operation even when external AI services experience issues.

CHAPTER 7

CONCLUSION

Abstractor AI successfully achieves its primary objective of providing accessible, high-quality document summarization through an intelligent, user-friendly web application. The project demonstrates that advanced AI capabilities can be delivered without prohibitive costs or complex infrastructure through thoughtful architecture and strategic use of cloud services.

The dual-algorithm approach combining state-of-the-art BART transformer models with classical TF-IDF extractive methods proves highly effective, delivering superior quality when AI services are available while maintaining functionality through intelligent fallback. This hybrid methodology addresses a critical limitation of pure AI-dependent systems, ensuring users receive summarization results regardless of external service status.

Multi-format support for PDF, DOCX, and TXT files significantly enhances usability by eliminating conversion requirements and streamlining workflows. Users can directly process documents in their native formats, reducing friction and improving adoption. The robust text extraction implementations handle various document structures and edge cases effectively, providing reliable processing across diverse content types.

The React-based user interface successfully delivers an intuitive experience requiring minimal learning curve. Features such as real-time validation feedback, drag-and-drop file upload, and responsive design contribute to high usability across desktop and mobile devices. Comprehensive history management with persistent storage adds significant value by enabling users to track and reference past summarizations.

Performance testing confirms the system meets target processing times for typical documents while maintaining acceptable performance for larger inputs through intelligent chunking algorithms. The scalable architecture built on modern web technologies supports concurrent users and provides clear paths for horizontal scaling as demand grows.

From a technical perspective, the project validates several important architectural decisions. The separation of frontend and backend components through RESTful APIs enables independent development and deployment. The use of established technologies (React, Flask, Python) rather than experimental frameworks ensures long-term maintainability and community support. The stateless design eliminates database complexity while localStorage provides adequate client-side persistence for history management.

Security and privacy considerations are appropriately addressed through in-memory file processing, environment-based API token management, and comprehensive input validation. The open-source nature of the project promotes transparency and allows users to verify security practices or deploy locally for sensitive content.

The project successfully demonstrates feasibility across technical, operational, and economic dimensions. All objectives defined in the initial requirements have been met or exceeded, validating the original project scope and approach. The resulting system provides genuine value to users by saving time, improving information access, and democratizing access to advanced NLP capabilities.

CHAPTER 8

FUTURE ENHANCEMENTS

8.1 MULTI-STYLE SUMMARIES

The system supports multiple summary styles so users can choose how information lands. One option is bullet-point summaries, where content is distilled into clear, scannable points instead of a single paragraph. Users can decide how many bullets they want, such as three, five, or ten, allowing control over depth and brevity. These bullet points emphasize key facts, decisions, and action items, making them ideal for quick understanding. This style is especially valuable for business users who need ready-to-use content for presentations, meetings, or executive reports where clarity and speed matter.

In addition, a headline and executive summary mode is provided for high-impact communication. This mode generates a one-line headline that captures the core topic or conclusion of the document, followed by a short executive summary of two to three sentences. The executive summary highlights the purpose of the document, its key findings, and the main outcomes. This format is designed for use in email introductions, report headers, dashboards, or anywhere a fast, high-level overview is required.

8.2 MULTI-LANGUAGE SUPPORT

The platform extends beyond English by supporting multiple languages for both input and output. The summarization backend is capable of automatically detecting the language of the uploaded text, while also allowing users to manually choose the desired output language. This enables use cases such as summarizing an English document into French or Spanish, or processing content originally written in other languages. As a result, Abstractor AI becomes practical for international teams and multilingual workflows.

Alongside language processing, the user interface itself is localized to ensure a seamless experience across regions. Frontend elements such as labels, buttons, error messages, and help hints are translated into supported languages. A language selector is included in the interface and stored as part of the user's preferences. The layout is designed to remain visually clean and usable even for languages with longer words or right-to-left writing systems, ensuring accessibility and consistency.

8.3 SUMMARIZATION EXPLAINABILITY

The system offers fine-grained control over summary length, moving beyond basic presets like short, medium, and long. Users can specify an approximate length using a slider or numeric input, such as a target word count or character limit. An estimated reading time is displayed for each generated summary to help users gauge effort and usability. Guardrails are implemented to prevent unrealistic or unsupported length requests, maintaining output quality and system stability.

An optional "Explain My Summary" mode adds transparency to the summarization process. In this view, the system highlights which parts of the original text most influenced the generated summary. Original sentences that strongly contributed to each summary sentence are visually marked, along with simple confidence or importance

scores. This feature helps users understand why certain information was included or excluded, increasing trust, interpretability, and confidence in AI-generated results. Personalization and user-specific preferences further refine how summaries are produced and presented.

8.4 AUTHENTICATION AND USER ACCOUNTS

To support personalization and advanced functionality, the system includes secure user registration and login. User accounts allow storage of summary history, preferences, and access to premium features. Authentication follows industry best practices, including password hashing and secure session or token-based mechanisms. Optional social login through providers such as Google or GitHub can be enabled for convenience. To enhance security, login endpoints are protected with rate limiting to reduce the risk of brute-force attacks.

The platform also implements role-based access control to manage capabilities across different user types. Standard users access the core summarization features with default usage limits. Power users may unlock batch processing, higher limits, or advanced analytics. Administrators have elevated permissions that allow them to manage users, monitor system usage, and configure global settings, ensuring smooth operation and governance.

8.5 IMPROVED LAYOUT AND NAVIGATION

The interface is redesigned with a multi-pane layout that displays the original text and the generated summary side by side. This allows users to compare content easily and understand how the summary maps to the source. Scrolling can be synchronized between panes for smoother navigation, and users can quickly switch between different summary lengths for the same document without reuploading it.

When a document is uploaded, the system provides a preview before summarization begins. This preview shows basic metadata such as file type, size, and page count, giving users clarity about what they are processing. Users can also select specific pages or sections to summarize instead of the entire document, enabling focused summaries and saving time when working with large files.

APPENDIX A:

BIBLIOGRAPHY

Lewis, M., et al. (2019). "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension." Facebook AI Research.

See, A., Liu, P.J., Manning, C.D. (2017). "Get To The Point: Summarization with Pointer-Generator Networks." Stanford University.

Nallapati, R., et al. (2016). "Abstractive Text Summarization using Sequence-to-sequence RNNs and Beyond." IBM Watson.

React Documentation (2024). "React: The Library for Web and Native User Interfaces." Meta Open Source.

Flask Documentation (2024). "Flask Web Development, one drop at a time." Pallets Projects.

Hugging Face Documentation (2024). "Transformers: State-of-the-art Machine Learning for PyTorch, TensorFlow, and JAX."

PyPDF2 Documentation (2024). "PyPDF2: A Pure-Python library built as a PDF toolkit."

python-docx Documentation (2024). "python-docx: Create and update Microsoft Word .docx files."

Vite Documentation (2024). "Vite: Next Generation Frontend Tooling." Evan You.

Axios Documentation (2024). "Axios: Promise based HTTP client for the browser and node.js."

APPENDIX B:

USER MANUAL

Getting Started

Accessing Abstractor AI: Open a modern web browser (Chrome 90+, Firefox 88+, Safari 14+, or Edge 90+). Navigate to the application URL (http://localhost:5000 for development). The application loads in under 2 seconds on standard broadband connections.

Text Input Method

Step 1: Ensure the Summarize tab is selected (active by default on first load). Step 2: Click inside the large textarea labeled "Enter your text here..." Step 3: Type or paste text content from any source (minimum 100 characters required). Step 4: Observe the character counter below the textarea showing current count with color indicator (gray: empty, orange: warning <100 chars, green: valid ≥100 chars). Step 5: Select desired summary length using buttons below input area (Short/Medium/Long). Step 6: Click the "Summarize" button when character counter shows green. Step 7: Wait for processing (typically 15-30 seconds) indicated by loading animation. Step 8: View generated summary in the output section below with statistics panel showing original length, summary length, and compression percentage.

File Upload Method

Step 1: Locate the file upload area below the text input (dashed border box). Step 2: Upload file using either method: Drag-and-drop: Drag PDF, TXT, or DOCX file from file explorer and drop onto the upload area (border highlights blue during drag-over), OR Click-to-browse: Click anywhere in the upload area to open file selection dialog, navigate to desired file, click Open. Step 3: Verify selected file appears with filename and size displayed. Step 4: To change file, click the remove/x button and select different file. Step 5: Select summary length preset (Short/Medium/Long). Step 6: Click "Summarize" button. Step 7: Wait for file processing and summarization (may take 30-60 seconds for large files). Step 8: Review generated summary and statistics in output section.

Understanding Summary Length Presets

Short Preset: Generates ultra-concise summaries approximately 60-80 characters, ideal for quick scanning and overview purposes, best for social media posts or very brief descriptions. Medium Preset (Recommended): Produces balanced summaries approximately 100-150 characters, suitable for most general purposes, provides good detail-to-brevity ratio. Long Preset: Creates detailed summaries approximately 150-250

characters, appropriate for comprehensive overviews, maintains more context and supporting details.

Viewing and Managing History

Accessing History: Click the "History" tab in the navigation bar. All previous summarizations appear in reverse chronological order (newest first). Each history card displays: Date and time of summarization, Source badge (Text or File indicator), Length preset used, Preview of original text (truncated), View details button. Viewing Details: Click "View Details" button on any history card. Modal popup displays full original text (or preview for very long texts) and complete generated summary. Close modal by clicking X button or clicking outside the modal. Clearing History: Click "Clear All History" button at top of History tab. Confirm deletion in the confirmation dialog. All history entries are permanently removed from browser storage.

Troubleshooting Common Issues

Summarize Button Disabled: Ensure text input has at least 100 characters or valid file is selected. Check character counter shows green valid state.

File Upload Failed: Verify file is PDF, TXT, or DOCX format. Confirm file size is under 16MB. Try renaming file to remove special characters.

Processing Takes Very Long: Large documents (>10,000 words) may take 60-90 seconds. Check internet connection is stable for AI processing. Consider using shorter preset to reduce processing time.

Summary Quality Poor: AI services may be temporarily unavailable (system uses fallback method). Try again in a few minutes when AI services resume. Ensure source document has clear, well-structured text.

History Not Saving: Check browser allows localStorage (not in incognito/private mode). Verify browser data not set to clear on exit. Try different browser if issue persists.

Tips for Best Results

Use well-formatted input text with clear sentences and paragraphs

For very long documents (>50,000 words), consider splitting into sections

Choose appropriate length preset based on intended use (Short for overviews, Long for detailed summaries)

Review statistics to understand how much compression was achieved

Save important summaries outside the system (copy to clipboard) as history is limited to 20 entries

Use the fallback TF-IDF method intentionally by disconnecting internet for fully local processing

2.3 TOOLS AND TECHNOLOGIES USED

2.3.1 FRONT END

React 19.2.1: Modern JavaScript library for building interactive user interfaces using component-based architecture, virtual DOM for efficient rendering, hooks for state management, and extensive ecosystem support.

Vite 7.2.6: Next-generation frontend build tool providing instant server start, lightning-fast Hot Module Replacement, optimized production builds using Rollup, and ES module support.

Axios 1.13.2: Promise-based HTTP client for API communication with automatic JSON transformation, timeout configuration, FormData support for file uploads, and comprehensive error handling.

2.3.2 BACK END

Python 3.11: Core programming language chosen for rich NLP ecosystem, excellent AI/ML integration, strong string manipulation, and widespread adoption.

Flask 3.0.0: Lightweight WSGI web framework with decorator-based routing, built-in request handling, JSON support, and minimal boilerplate code.

PyPDF2 3.0.0: Pure-Python PDF library for text extraction without external dependencies, supporting various PDF versions and encrypted documents.

python-docx 1.0.0: Library for Microsoft Word document processing, extracting paragraph text from DOCX files (Office 2007+).

Requests 2.31.0: HTTP library for external API calls to Hugging Face, providing authentication handling, timeout management, and JSON processing.

Hugging Face Transformers API: Provides access to facebook/bart-large-cnn model with ~400 million parameters trained on CNN/DailyMail dataset for abstractive summarization.